

Rony Zeenaldeen

Back-End Engineer · AI / ML Engineer

ronizenalden@gmail.com | [LinkedIn](#) | [GitHub](#) | [Portfolio](#)

SUMMARY

Back-End & AI/ML Engineer (4+ years) building production-grade systems across scalable APIs, event-driven platforms, and applied AI (agentic RAG, NLP, computer vision). Polyglot back-end developer — expert in Django and Go, with FastAPI powering dedicated AI-serving backends, and end-to-end ML pipelines from training to deployment. Proven impact: 40× faster queries, 60–70% API speedup via Redis caching, and 65% reduction in database load. Builds event-driven, fully observable infrastructure — Apache Kafka and RabbitMQ for messaging, Kubernetes for container orchestration, Prometheus/Grafana for monitoring, and pgBackRest/pgBadger for database backup and reliability. Experienced in deploying transformer-based NLP systems, hybrid RAG pipelines with vector search, and high-accuracy computer-vision models (AUC-ROC 0.997) for medical imaging.

TECHNICAL SKILLS

Back-End: Django, Django REST Framework, FastAPI, Go (Golang), Node.js, REST API Design, GraphQL, WebSockets

AI / ML: Python, TensorFlow, PyTorch, Keras, Scikit-learn, XGBoost, EfficientNetV2, Facebook Prophet, Transformer Models

Databases: PostgreSQL, Redis, MongoDB, MySQL, Firebase Firestore

Messaging & Streaming: RabbitMQ, Apache Kafka

Cloud & AWS: IAM, S3, RDS, SES, Route 53, Elastic Beanstalk

DevOps & Infrastructure: Docker, Kubernetes (Basics), NGINX, GitHub Actions, CI/CD, Linux

Monitoring & Reliability: Prometheus, Grafana, pgBackRest, pgBadger

Mobile: Flutter, Dart, GetX, BLoC, Firebase

Programming Languages: Python, Go, TypeScript, Dart, SQL, JavaScript

PROFESSIONAL EXPERIENCE

AI / ML Engineer · Freelance · Nov 2024 – Present

Designed and deployed production-grade AI systems — from agentic RAG to NLP and computer vision — architecting the event-driven, observable infrastructure that runs them in production.

- Built and deployed a Retrieval-Augmented Generation (RAG) system with document ingestion, embeddings, vector search, and LLM-based generation for context-aware QA.
- Developed a production NLP platform with real-time inference and batch pipelines for sentiment analysis, complaint detection, and classification.
- Built a chest X-ray classifier (EfficientNetV2S) achieving AUC-ROC 0.997 and F1 0.95 — deployed as a public Django REST API.
- Developed a sales forecasting pipeline (Facebook Prophet) on 1M+ transactions, enabling data-driven inventory and staffing decisions.
- Designed an HR attrition model (87% accuracy), estimating \$225K–\$450K annual retention savings.
- Built event-driven AI infrastructure — FastAPI inference microservices decoupled from the core app, Apache Kafka/RabbitMQ pipelines for streaming and batch AI workloads, and full observability (Prometheus/Grafana) with pgBackRest-based backup and point-in-time recovery.
- Deployed and managed services using AWS (S3, RDS, IAM, SES, Elastic Beanstalk, Route 53) and Kubernetes-orchestrated containers for scalable, reliable production environments.

Back-End Developer · Freelance · Jan 2022 – Present

Designed and delivered scalable, polyglot backend systems and event-driven architectures for production applications.

- Built and shipped REST APIs — using Django REST Framework and Go — with JWT/OAuth2 authentication, versioning, and OpenAPI documentation, plus GraphQL endpoints for flexible client queries.
- Optimized system performance: reduced PostgreSQL query time from 2000ms to <50ms and improved API latency by 60–70% using Redis caching.
- Designed real-time systems (WebSockets, Django Channels, Redis) for live ride-matching and location streaming.
- Integrated payment systems (Stripe, PayPal, Razorpay) with secure webhook handling and processing.
- Introduced Go microservices alongside Django for latency-critical, high-concurrency components (WebSocket hubs, event consumers), with RabbitMQ/Kafka decoupling asynchronous processing and pgBackRest/pgBadger strengthening PostgreSQL backup and query analysis.
- Deployed backend services using Docker, Kubernetes, NGINX, and AWS infrastructure for high availability and scalability.

KEY PROJECTS

NeuroSeek AI — Agentic AI Research Platform (LIVE) · 2026 – Present [↗ Link](#)

Self-hosted research assistant that plans multi-step search queries, crawls and embeds web sources, and answers via hybrid RRF-fused RAG (pgvector semantic + keyword search) — streaming a fully cited report token-by-token.

- Implemented an agentic “gap-driven” re-search loop: the system audits its own answer coverage, identifies gaps, and autonomously re-searches before finalizing a report.
- Engineered a polyglot backend — Django 5 + DRF + Celery/Redis for the application layer, with dedicated Go services for a real-time WebSocket streaming hub and a Kafka consumer powering the analytics pipeline.
- Deployed cloud-native via Docker Compose and Kubernetes (Helm, k3s/Rancher-ready) with full observability — Apache Kafka (KRaft) event streaming, Prometheus/Grafana alerting, and pgBackRest (PITR) + pgBadger for database reliability — and hardened with SSRF protection, JWT rotation/blacklisting, and rate limiting.

Tech: Python · Django 5 · DRF · Celery · Redis · Go · PostgreSQL 16 · pgvector · Apache Kafka · Ollama · Next.js 14 · TypeScript · Docker · Kubernetes · Prometheus · Grafana · pgBackRest

PulmoSight — Doctor-in-the-Loop Chest X-Ray AI (LIVE) · 2026 – Present [↗ Link](#)

Self-hostable clinical AI system that classifies 18 chest pathologies with Grad-CAM heatmaps and drafts a structured radiology report grounded strictly in the classifier’s confirmed findings, built for clinics that can’t afford commercial radiology AI.

- Built a redaction-filter safety layer that strips any AI-generated finding the classifier didn’t confirm — no report reaches a patient without a doctor’s review and sign-off via an auditable approval workflow.
- Designed the detection pipeline: an ensemble of fine-tuned DenseNet-121 classifiers with per-disease F1-tuned thresholds and a grounded VLM report writer (switchable Qwen2-VL-2B / Phi-3.5-vision / MAIRA-2 backend).
- Architected a polyglot backend — Django 5 + DRF for the clinical app and audit logging, a separate FastAPI inference service, and a Go-based Telegram bot (RabbitMQ-decoupled from doctor approval) delivering a bilingual (Arabic RTL / English) report via QR + single-use PIN onboarding, consent capture, and a branded PDF.

Tech: Python · Django 5 · DRF · FastAPI · Go · RabbitMQ · PostgreSQL 16 · PyTorch · Grad-CAM · Docker · Telegram Bot API

ROTECHAI — AI + Software Studio Platform (LIVE) · 2026 – Present [↗ Link](#)

Self-hosted, multilingual (EN/AR/NL/DE) marketing site paired with a live AI product — real-time text classification and a RAG chat assistant for services, pricing, and process questions.

- Right-sized the entire AI layer to a single ~400MB multilingual embedding model (fastembed / ONNX Runtime, no PyTorch, lazy-loaded) with a hybrid cosine + keyword RAG over an in-memory knowledge base — no GPU or paid LLM API required.
- Designed the backend as Django 5 + DRF for the main application paired with an independently restartable FastAPI microservice for AI inference, so an AI-service restart never takes the site down.
- Built a Next.js 16 (App Router) + React 19 + TypeScript frontend across 4 locales with full RTL Arabic support and SEO best practices (structured data, hreflang, sitemaps).
- Hardened and deployed the full stack behind a single NGINX edge with HSTS, CSRF/cookie hardening, brute-force lockout, rate limiting, and environment-only, fail-closed secrets.

Tech: Python · Django 5 · DRF · FastAPI · PostgreSQL 16 · Redis · Next.js 16 · React 19 · TypeScript · Docker · NGINX

Local RAG Document QA System — LLM Retrieval-Augmented Generation · 2026 [↗ Link](#)

- Built a fully local Retrieval-Augmented Generation (RAG) pipeline for document-based QA using LLMs, embeddings, and vector search.
- Implemented document ingestion, chunking, embedding storage, and semantic retrieval for accurate context-aware responses.
- Designed end-to-end pipeline: query → vector similarity search → context injection → LLM answer generation.
- Optimized for privacy and offline usage with local model inference.

Tech: Python · LangChain / LlamaIndex · Vector DB · Transformers · Embeddings · Django

EDUCATION

B.Sc. Informatics Engineering — Software & Applications Development

Al-Shahba Private University, Aleppo, Syria · Sep 2020 – Jun 2025

CERTIFICATIONS

- Artificial Intelligence Essentials V2 — IBM (2025)
- Data Engineering Professional — IBM (2025)
- Cisco Certified Network Associate (CCNA) — New Horizons (2023)
- Project Management Professional (PMP v6) — Project International (2021)

LANGUAGES

- Arabic — Native
- English — B2-C1 (Proficient)